

You Don't Need **RAG**

Paste a list of URLs, get one clean text file or a structured RAG-ready ZIP — whichever your knowledge base actually needs. A tool built on a simple bet: most "you need RAG" problems fit in a context window.

Next.js 15

React 19

Python scraper

No vector DB

No embeddings

LIVE DEMO

rag.huxiaoheng.com

SOURCE

github.com/huxiaoheng44/You-Don-t-Need-RAG

WHY THIS EXISTS

RAG has become the default answer to everything

Need to query your docs? RAG. Answer questions about a website? RAG. Give context to a chatbot? RAG. The pattern has real merits — but a long tail of real costs.

Chunking is lossy	Splitting documents into fixed-size chunks destroys context that spans paragraphs. The model only sees fragments.
Retrieval is imperfect	Embedding similarity does not equal semantic relevance — the model confidently answers from the wrong chunk anyway.
Operationally heavy	An embedding model, a vector database, an ingestion pipeline, a retrieval layer — every piece can fail or drift silently.
Evaluation is hard	No single obvious quality metric. Hallucinations can trace back to any layer of the stack.
Latency adds up	Every query needs an embedding round-trip plus a vector search before the LLM sees a token.

"The engineering tendency is to reach for the most sophisticated tool available. The complexity was the overhead, not the solution."

THE REAL FIX

Modern LLMs already fit the whole knowledge base

GPT-4o, Claude, Gemini 1.5 Pro now support 128K to 1M+ token context windows. A 300 KB text file is roughly 75K tokens. No chunking. No retrieval. No pipeline. Just paste.

128K–1M

TOKENS OF CONTEXT, MODERN LLMS

75K

TOKENS IN A 300 KB TEXT FILE

STEP 1

Scrape the URLs, extract clean readable content



STEP 2

Measure the actual size of the scraped content

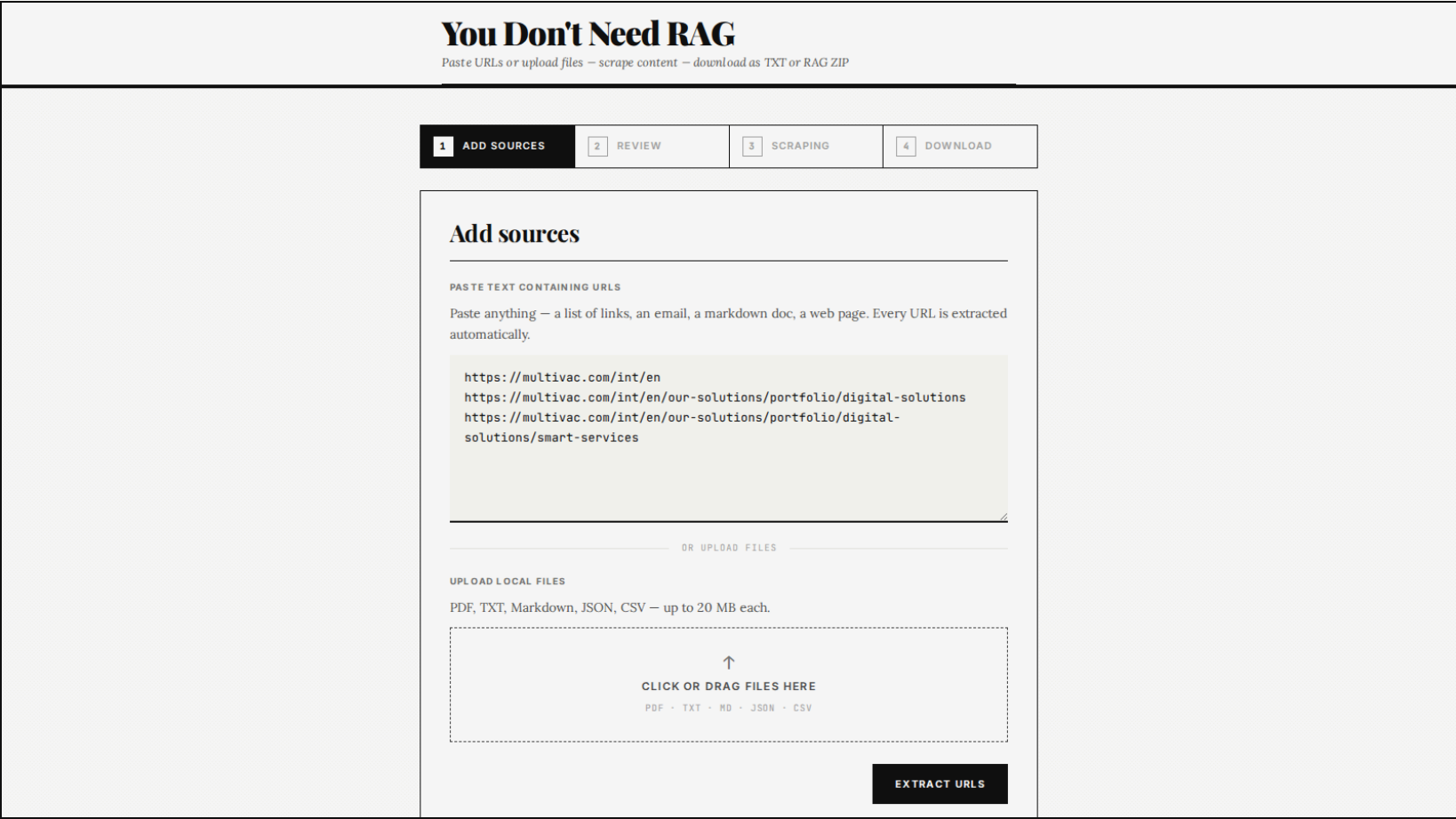


DECISION

Fits a context window? → paste the plain text file.
Genuinely large / live-updating / multi-tenant? → export the RAG ZIP.

HOW IT WORKS

Four steps, no configuration

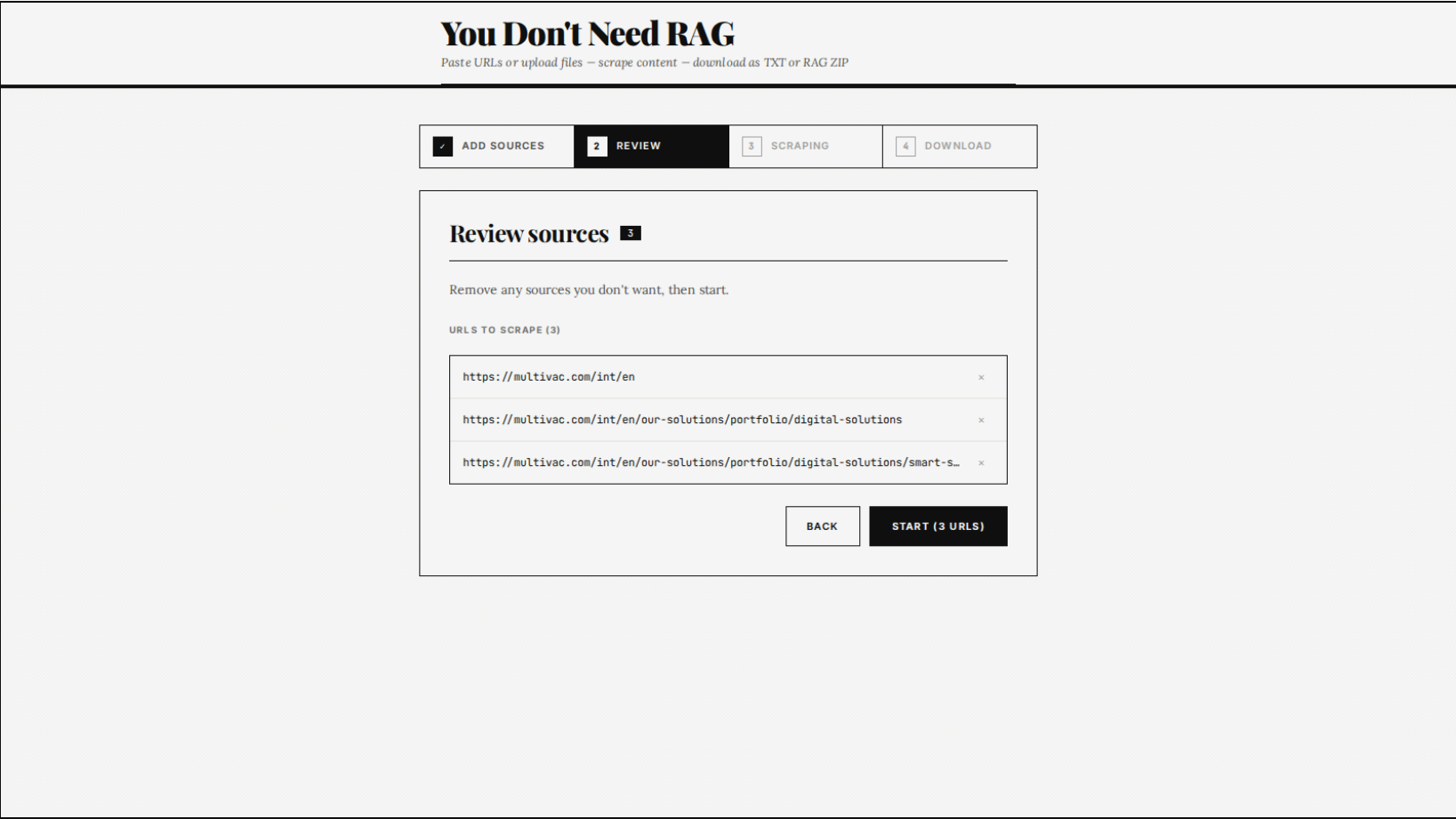


01 – ADD SOURCES

Paste anything with links in it

A list, an email, a markdown doc — every URL is extracted automatically. Local files (PDF, TXT, MD, JSON, CSV) work too.

Steps 3–4 (scraping, then choosing plain text vs. RAG ZIP based on actual content size) are covered on the next slide.



02 – REVIEW

Remove what you don't want

URLs and uploaded files are listed separately before scraping starts — nothing runs until you confirm.

OUTPUT

Two formats, recommended by actual size

The tool measures the real scraped content and tells you which format fits — it doesn't make you guess.

Plain Text

knowledge_base.txt — paste directly into any LLM

```
≡≡≡ Page Title ≡≡≡
URL: https://example.com
Scraped: 2026-01-01T00:00:00Z

Full page content here...

---

≡≡≡ Next Page ≡≡≡
...
```

RECOMMENDED WHEN IT FITS A CONTEXT WINDOW

RAG ZIP

knowledge_base.zip — ready for an embedding pipeline

```
knowledge_base.zip
├─ manifest.json  # id, url, title, char_count
└─ pages/
    ├─ example-com-abc123.json
    ├─ example-com-docs-def456.json
    └─ ...
```

RECOMMENDED WHEN RAG IS ACTUALLY THE RIGHT TOOL

NOT ANTI-RAG

When you actually do need RAG

This tool isn't anti-RAG. It's pro-pragmatism. RAG is the right call when:

Your knowledge base is genuinely large — millions of tokens, not thousands	You need sub-second retrieval over a live-updating corpus
You want to retrieve across many different users' private datasets	Your queries are specific enough that full-document context would mostly be noise

"For everything else — try the plain text file first."

STACK

Everything that shipped

Next.js 15 App Router, React 19	Python scraper requests, BeautifulSoup, markdownify, pdfplumber	File-based jobs Async scraping jobs in data/jobs/<uuid>
Plain text output Single concatenated .txt, paste-ready	RAG ZIP output manifest.json + per-page JSON via zipfile	No vector DB The point of the project — zero infra by default

LIVE DEMO

rag.huxiaoheng.com

SOURCE

github.com/huxiaoheng44/You-Don-t-Need-RAG